

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №4
«Технический дизайн микросервисной архитектуры»

Выполнил:

Аплеев Дмитрий

К3339

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Разделить монолит на микросервисы по принципу database per service, спроектировать связи между ними, разделить базу данных и описать межсервисные эндпоинты в OpenAPI.

Вариант 5. Сервис для обмена рецептами и кулинарных блогов.

Ход работы

1. Сервисы

Сервис	Порт	Что делает	Таблицы
bff-service	3004	единая точка входа, пересылает запросы и собирает ответы	нет
user-service	3001	регистрация, вход, профили	user
recipe-service	3002	рецепты, ингредиенты, поиск	recipe, ingredient
social-service	3003	комментарии, лайки, сохранения, подписки	comment, like, saved_recipe, subscription

Таблица 1. Сервисы



Рисунок 1. Архитектура

2. Базы данных

У каждого сервиса своя база, в чужую он не ходит. У bff-service базы нет. Внешние ключи между разными базами невозможны, поэтому такие поля стали просто числами. Проверку делает код. Перед лайком,

комментарием и сохранением social-service спрашивает recipe-service, есть ли рецепт. Перед подпиской он спрашивает user-service, есть ли пользователь. Автор и пользователь берутся из токена.

База	Сервис	Таблицы
user_db	user-service	user
recipe_db	recipe-service	recipe, ingredient
social_db	social-service	comment, like, saved_recipe, subscription

Таблица 2. Разделение базы

3. Взаимодействие

Клиент знает только адрес bff-service. Для него API осталось таким же, как у монолита. BFF пересылает запрос в нужный сервис, а если данные лежат в разных сервисах, собирает ответ сам. Например, страница рецепта берёт рецепт из recipe-service, имя автора из user-service и число лайков из social-service. Сервисы общаются по HTTP через служебные адреса /api/v1/internal/.... Токен проверяет каждый сервис сам общим секретным ключом.

4. Межсервисные эндпоинты

Эндпоинт	Кто вызывает	Зачем
GET /v1/internal/users?ids=1,2	BFF	имена авторов для списков
GET /v1/internal/users/{id}	social	есть ли пользователь
GET /v1/internal/recipes?ids=1,2	BFF	карточки сохранённых рецептов
GET /v1/internal/recipes/{id}	social	есть ли рецепт
GET /v1/internal/likes/count?recipe_ids=1,2	BFF	число лайков
GET /v1/internal/users/{id}/followers-count	BFF	число подписчиков
DELETE /v1/internal/recipes/{id}	recipe	удалить лайки, комментарии и сохранения удалённого рецепта

Таблица 3. Межсервисные эндпоинты

Пример ответа.

```
GET /api/v1/internal/likes/count?recipe_ids=1,2
200 OK { "1": 3, "2": 0 }
```

Recipes — внутреннее API микросервисов 1.0.0 OAS 3.0

Эндпоинты для взаимодействия сервисов между собой (не для клиента). Клиент обращается только к bff-service; внутренние эндпоинты вызываются bff-service и сервисами друг у друга.

Любой запрос к сервису (не только к /v1/internal) должен содержать заголовок X-Internal-Key с общим секретным ключом INTERNAL_API_KEY. Без него сервис отвечает 403 — так обращение к сервисам в обход BFF невозможно.

Servers

http://localhost:3001/api - user-service

Authorize

user-service

Порт 3001, база user_db

GET

/v1/internal/users

Короткие данные нескольких пользователей

GET

/v1/internal/users/{id}

Проверка существования пользователя

recipe-service

Порт 3002, база recipe_db

GET

/v1/internal/recipes

Карточки нескольких рецептов

GET

/v1/internal/recipes/{id}

Проверка существования рецепта

social-service

Порт 3003, база social_db

DELETE

/v1/internal/recipes/{id}

Удалить данные, связанные с рецептом

GET

/v1/internal/likes/count

Количество лайков у нескольких рецептов

GET

/v1/internal/users/{id}/followers-count

Количество подписчиков пользователя

Рисунок 2. Межсервисные эндпоинты в Swagger

5. Ошибки

Ситуация	Ответ
объект не найден	404
запрос к сервису в обход BFF	403
сервис не отвечает	BFF возвращает 503
ошибка в запросе клиента	сервис отвечает 400, 403 или 409, BFF передаёт ответ как есть

Таблица 4. Ошибки

6. Шаги разделения

- создать четыре сервиса на том же boilerplate и три базы
- разнести модели по сервисам и убрать связи между разными базами
- перенести контроллеры, каждый сервис отдаёт только свои данные

- добавить служебные эндпоинты `internal`
- сделать `bff-service` с тем же API, что у монолита
- проверить старым набором запросов, что ответы не изменились

Вывод

Спроектировано разделение монолита на четыре сервиса с отдельными базами, описаны связи между ними, служебные эндпоинты и ошибки. Файл спецификации `internal-api.yaml`.